Week1

- Theoretical RAG
- RAG by Langchain
- RAG Evaluation Frameworks

Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

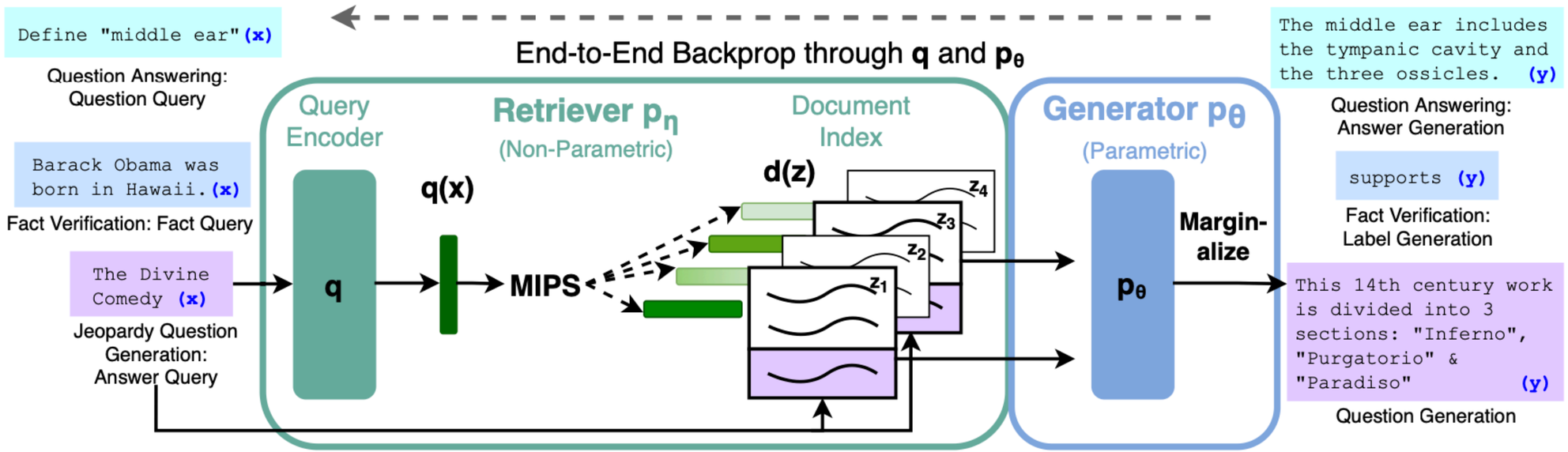


Figure 1: Overview of our approach. We combine a pre-trained retriever (*Query Encoder* + *Document Index*) with a pre-trained seq2seq model (*Generator*) and fine-tune end-to-end. For query x, we use Maximum Inner Product Search (MIPS) to find the top-K documents z_i. For final prediction y, we treat z as a latent variable and marginalize over seq2seq predictions given different documents.

RAG-Sequence Model The RAG-Sequence model uses the same retrieved document to generate the complete *sequence*. Technically, it treats the retrieved document as a single latent variable that is marginalized to get the seq2seq probability $p(y|x)$ via a top-K approximation. Concretely, the top K documents are retrieved using the retriever, and the generator produces the output sequence probability for each document, which are then marginalized,

$$p_{\text{RAG-Sequence}}(y|x) \approx \sum_{z \in \text{top-}k(p(\cdot|x))} p_\eta(z|x) p_\theta(y|x,z) = \sum_{z \in \text{top-}k(p(\cdot|x))} p_\eta(z|x) \prod_i^N p_\theta(y_i|x,z,y_{1:i-1})$$

RAG-Token Model In the RAG-Token model we can draw a different latent document for each target *token* and marginalize accordingly. This allows the generator to choose content from several documents when producing an answer. Concretely, the top K documents are retrieved using the retriever, and then the generator produces a distribution for the next output token for each document, before marginalizing, and repeating the process with the following output token, Formally, we define:

$$p_{\text{RAG-Token}}(y|x) \approx \prod_i^N \sum_{z \in \text{top-}k(p(\cdot|x))} p_\eta(z|x) p_\theta(y_i|x,z,y_{1:i-1})$$

Marginallizing z (the document) and then applying auto-regressive factorization (current token depends on previous tokens)

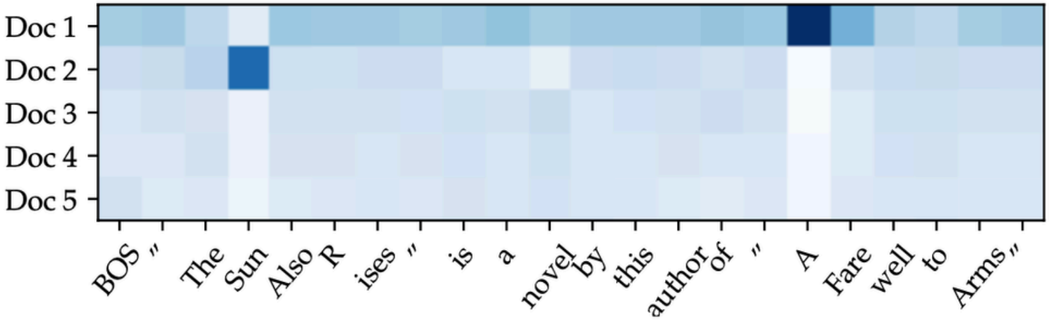
Evaluations

- Exact matching (short-answer questions)
- Abstractive question answering (complex) => Bleu (precision based)and Rougue-L (recall-based)
- Jeopardy question generation (token > sequence)
- Fact Verification (Supported, refuted or not enough info), no retrievel supervision

For fact verification using word-overlap based retriever worked better

Document 1: his works are considered classics of American literature ... His wartime experiences formed the basis for his novel **"A Farewell to Arms"** (1929) ...

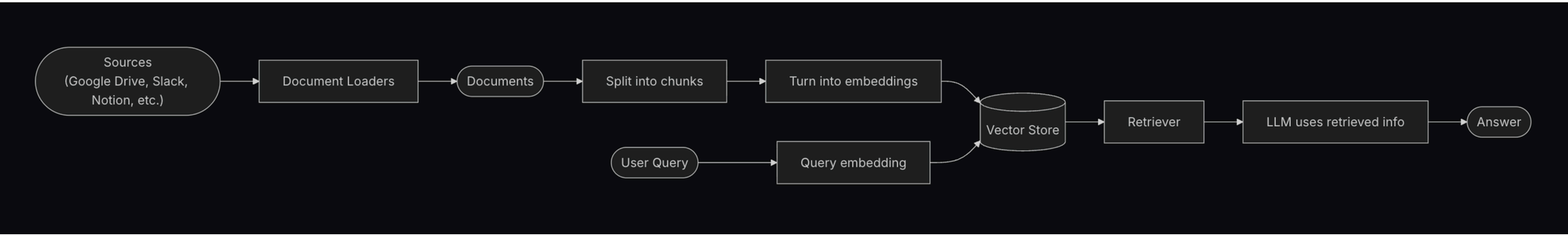
Document 2: ... artists of the 1920s "Lost Generation" expatriate community. His debut novel, **"The Sun Also Rises"**, was published in 1926.

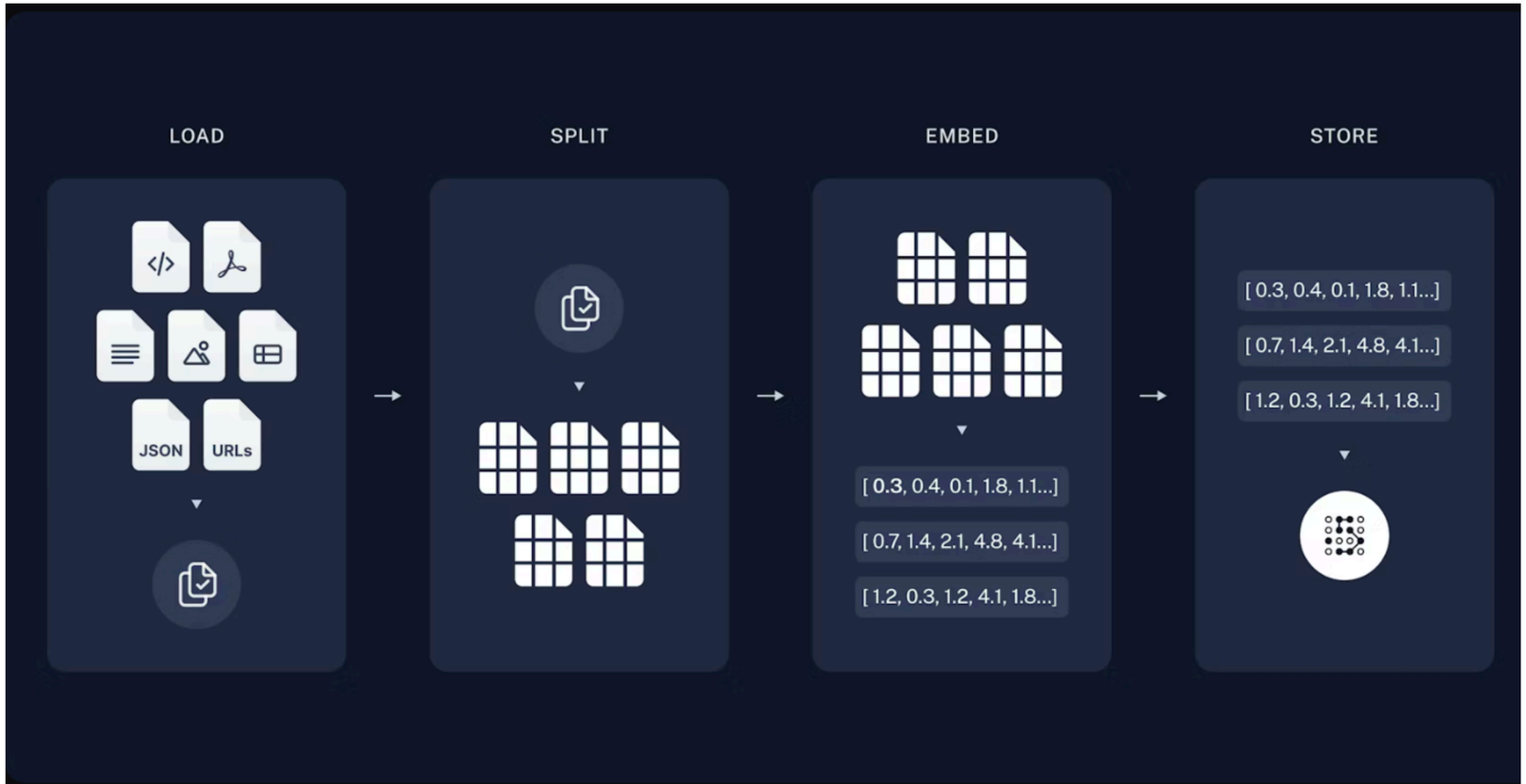


Rag-token

$$\frac{\partial \mathcal{L}}{\partial \theta} = \sum_{z} \underbrace{p(z \mid x, y_i, y_{-i})}_{\text{posterior}} \cdot \frac{\partial}{\partial \theta} \log p(y_i \mid x, z, y_{-i})$$

Rag by Langchain





Agentic Rag

RAG chains

In the above **agentic RAG** formulation we allow the LLM to use its discretion in generating a **tool call** to help answer user queries. This is a good general-purpose solution, but comes with some trade-offs:

✅ Benefits	⚠️ Drawbacks
Search only when needed – The LLM can handle greetings, follow-ups, and simple queries without triggering unnecessary searches.	**Two inference calls** – When a search is performed, it requires one call to generate the query and another to produce the final response.
Contextual search queries – By treating search as a tool with a `query` input, the LLM crafts its own queries that incorporate conversational context.	**Reduced control** – The LLM may skip searches when they are actually needed, or issue extra searches when unnecessary.
Multiple searches allowed – The LLM can execute several searches in support of a single user query.	

Agentic Rag

```python
@tool(response_format="content_and_artifact")
def retrieve_context(query: str):
    """Retrieve information to help answer a query."""
    retrieved_docs = vector_store.similarity_search(query, k=2)
    serialized = "\n\n".join(
        (f"Source: {doc.metadata}\nContent: {doc.page_content}")
        for doc in retrieved_docs
    )
    return serialized, retrieved_docs
```

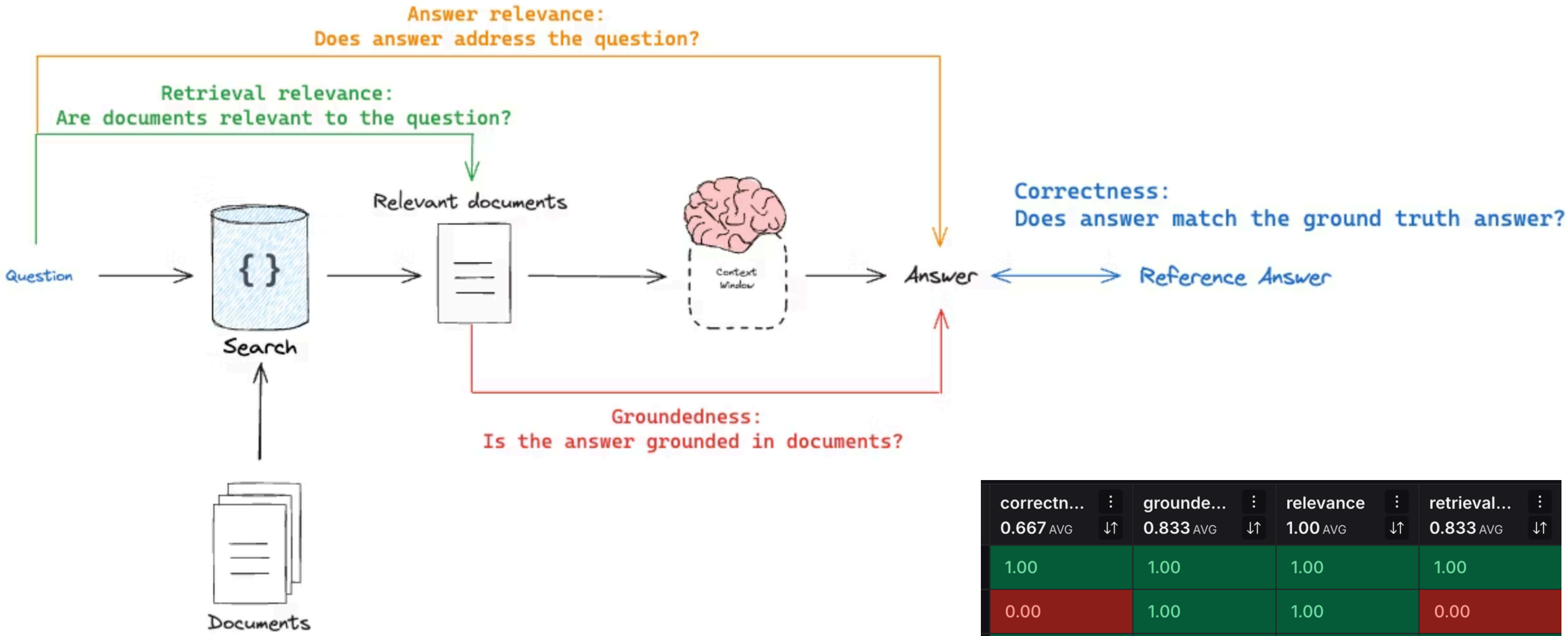
```python
@dynamic_prompt
def prompt_with_context(request: ModelRequest) -> str:
    "Inject context into state messages."
    last_query = request.state["messages"][-1].text
    retrieved_docs = vector_store.similarity_search(last_query)

    docs_content = "\n\n".join(doc.page_content for doc in retrieved_docs)

    system_message = (
        "You are a helpful assistant. Use the following context in your response:"
        f"\n\n{docs_content}"
    )

    return system_message
```

Rag Eval By Langchain

- Code
- Human Eval
- LLM-as-a-judge

Evaluators (LLM is the judge)

- Correctness
- Relevance
- Groundedness
- Retrieval Relevance



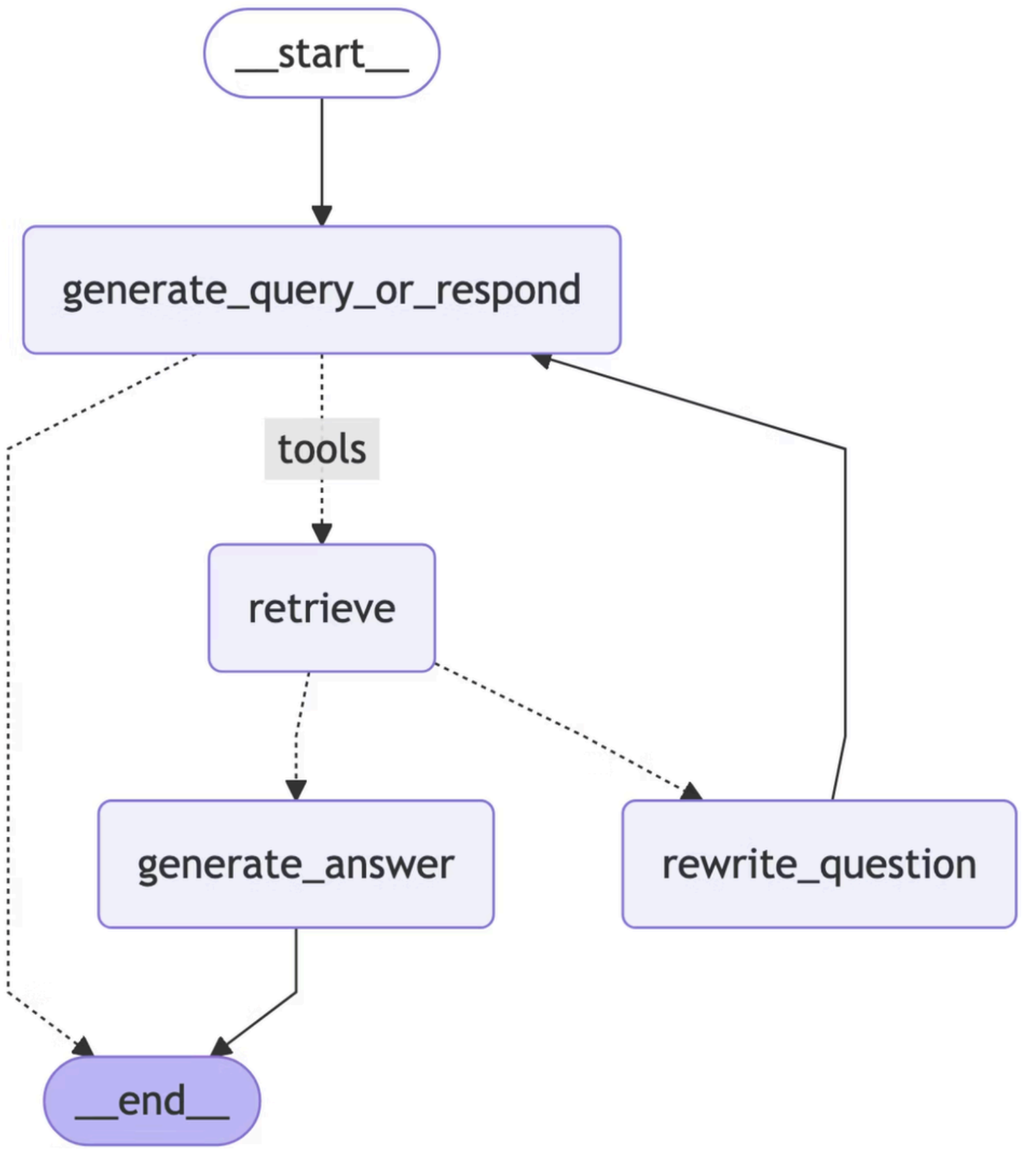
Evaluators

correctn... 0.667 AVG	grounde... 0.833 AVG	relevance 1.00 AVG	retrieval... 0.833 AVG
1.00	1.00	1.00	1.00
0.00	1.00	1.00	0.00
1.00	1.00	1.00	1.00
1.00	0.00	1.00	1.00
0.00	1.00	1.00	1.00
1.00	1.00	1.00	1.00

How to achieve better RAG?

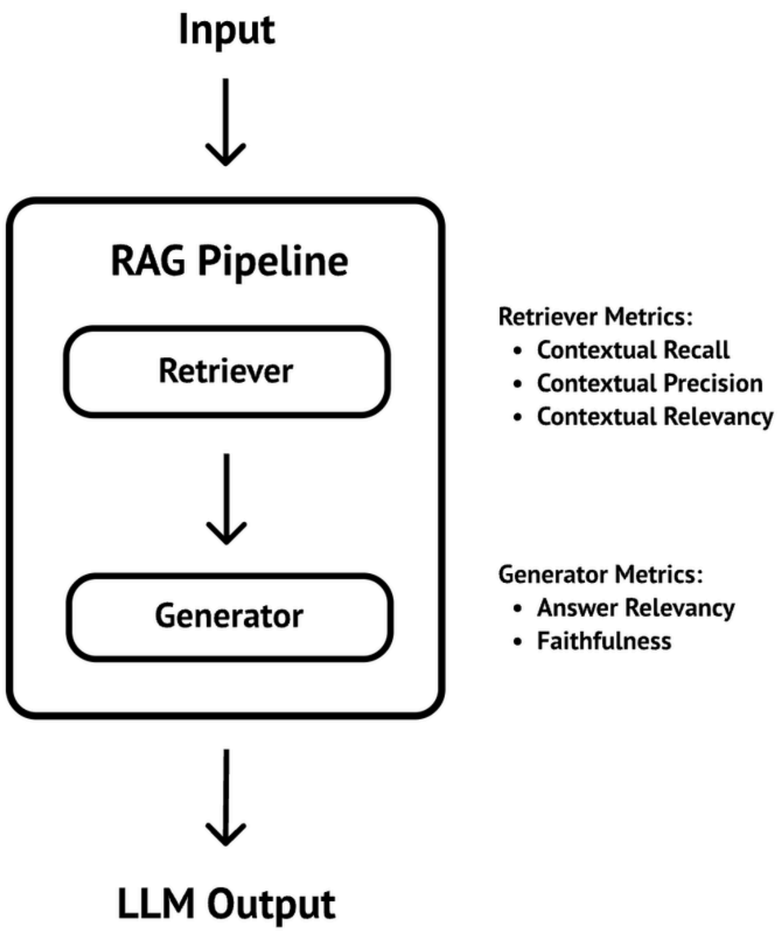
- Embedding model should be familiar with the nuances in the domain, fine-tune embedding model?
- Use re-ranker?
- Optimize Hyperparms (heuristics?)

- Fine-tune the LLM
- Change prompt template



Grade document relevance and rewrite query

DeepEval



retrievel step hyperparams:

- embedding model (query)
- top-k
- prompt-template
- chunk size
- [temperature]

https://deepeval.com/guides/guides-rag-evaluation

The `ContextualRecallMetric` score is calculated according to the following equation:

$$\text{Contextual Recall} = \frac{\text{Number of Attributable Statements}}{\text{Total Number of Statements}}$$

The `ContextualRecallMetric` first uses an LLM to extract all **statements made in the `expected_output`**, before using the same LLM to classify whether each statement can be attributed to nodes in the `retrieval_context`.

$$\text{Contextual Precision} = \frac{1}{\text{Number of Relevant Nodes}} \sum_{k=1}^{n} \left(\frac{\text{Number of Relevant Nodes Up to Position } k}{k} \times r_k \right)$$

$$\text{Contextual Relevancy} = \frac{\text{Number of Relevant Statements}}{\text{Total Number of Statements}}$$

Although similar to how the `AnswerRelevancyMetric` is calculated, the `ContextualRelevancyMetric` first uses an LLM to extract all statements made in the `retrieval_context` instead, before using the same LLM to classify whether each statement is relevant to the `input`.

LLM as a judge

In precision evaluation, the order matters, as **data in the higher part of the context window contributes more**

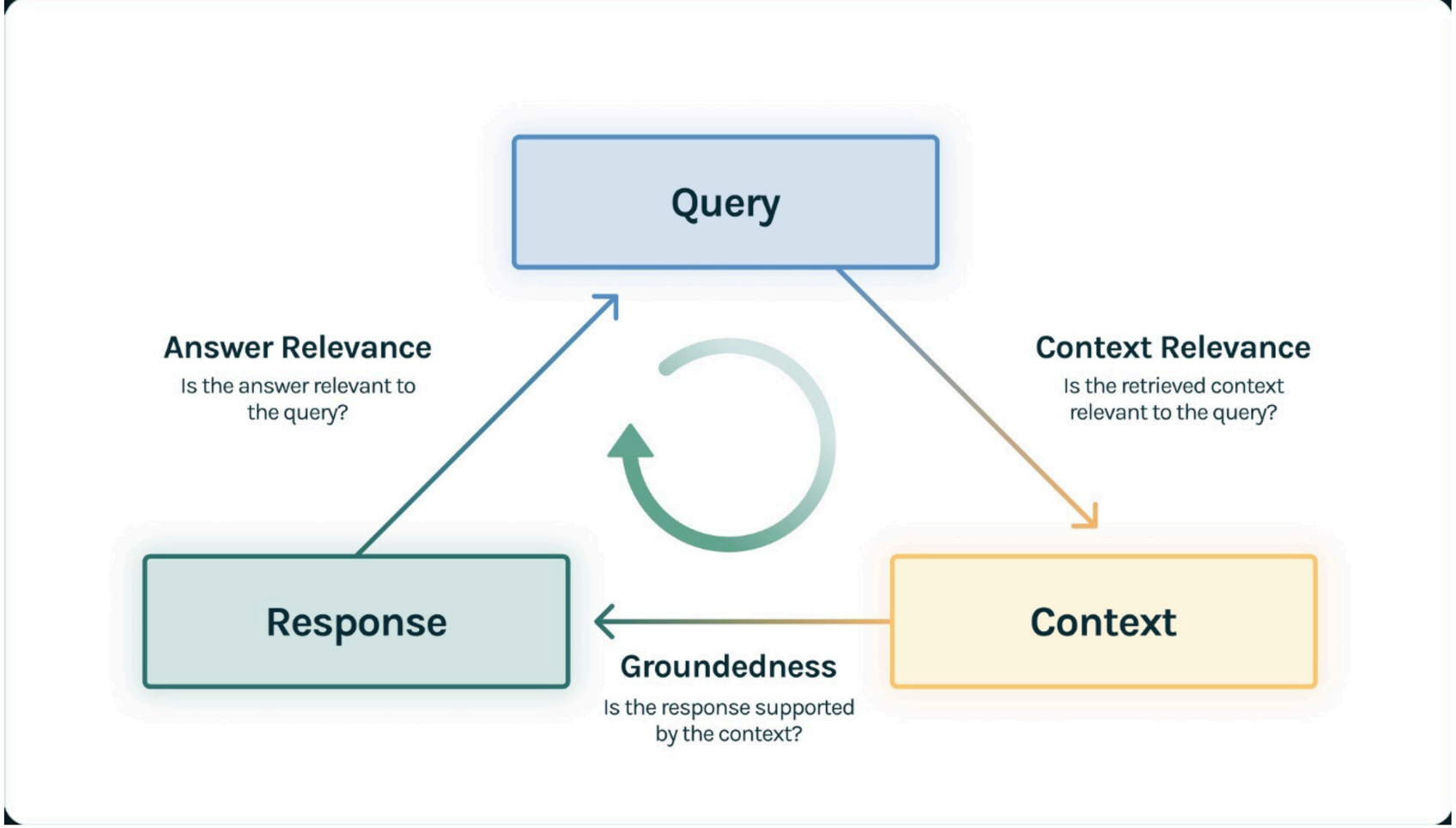
More relevant information being prioritized avoid downstream hallucination

Facebook Research Benchmark

https://github.com/facebookresearch/CRAG

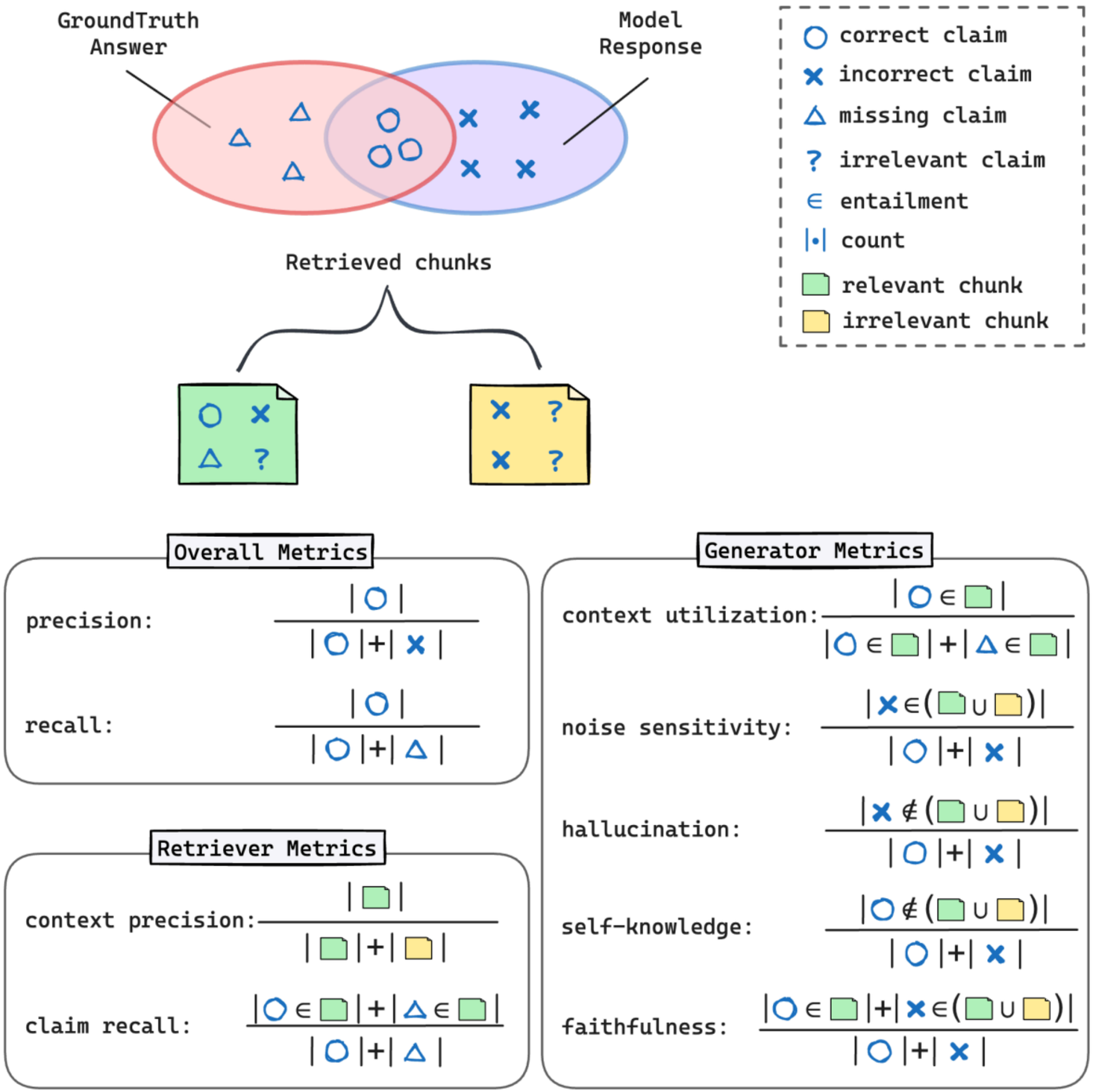
Dataset used in KDD-Cup 2024

Uses gpt-4 after some quick code checks
uses template score: 0/1, explanation: ...



trulens use llm as a judge

optionally **NLI** can be used for groundedness

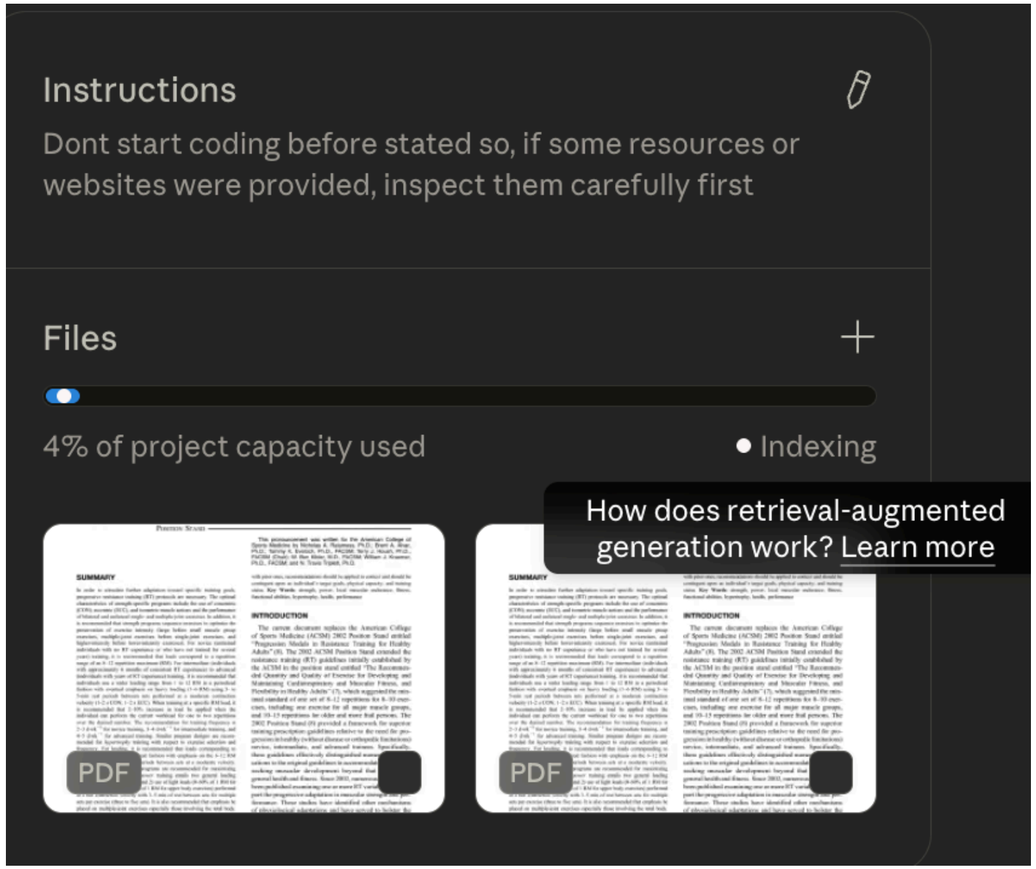


RAGCHECKER: A Fine-grained Framework for Diagnosing Retrieval-Augmented Generation

Need two functions:

1. Divide text into subclaims
2. Check whether a claim can be inferred from reference text

RAG helps augment the context window of LLMs



Next?

Check RAGas

Build a simple knowledge hub

Build simple RAG system

Use frameworks&metrics (one llm-as-judge and one with sth else)

Improve

What else?

NLI for analyzing contradicting knowledge in database